A Large-Scale Distributed Sorting Algorithm Based on Cloud Computing

Na Pang(✉), Dali Zhu, Zheming Fan, Wenjing Rong, and Weimiao Feng

Institute of Information Engineering,
Chinese Academy of Sciences, Beijing 100093, China
{pangna, zhudali, fanzheming, rongwenjing, fengweimiao}@iie.ac.cn

Abstract. Large amounts of data processing bring parallel computing into sharp focus. In order to solve the sorting problem of large-scale data in the era of internet, a large-scale distributed sorting algorithm based on cloud computing is proposed. The algorithm uses the ideas of quick-sort and merge-sort to sort and integrate the data on each cloud, which making best of clouds' computing and storage resources. Taking advantage of the idea of parallel computing, we can reduce the computing time and improve the efficiency of sorting. By evaluating algorithm's time complexity and organizing a simulation test, the effectiveness was verified.

Keywords: Cloud computing · Parallel algorithm · Quick sort · Merge sort · Distributed computing

1 Introduction

The increasingly development of Internet business makes total quantity of data which is to be processed grow exponentially. Massive data processing has become a pressing issue, especially for large-scale data sorting. Although there are many mature data sorting methods such as quick-sort, insertion-sort, merge-sort and shell-sort, these technologies can quickly achieve the sorting in case that the amount of data is small. The efficiency of these sorting algorithms would be greatly discounted when faced with large amount of data. The time performing the sorting is intolerable.

Some recent work has been proposed to improve existing algorithms to meet specific requirements. Big data process makes parallel computing get a lot of attention [1–3, 5]. Davidson et al. [4] design a high-performance parallel merge sort for highly parallel system to use more register communication. Minsoo and Dongseung [6] use multiple processors in parallel computing to perform merge-sort. The maximum speedup can reach in which P is the number of processors. Nanjesh et al. [7] aim to form a common single node model for both MPI and PVM which demonstrates the performance dependency of parallel merge sort on RAM of the nodes (desktop PCs) used in parallel computing. Christof et al. [8] propose adaptations of two especially classic sequential sorting algorithms such as bubble sort and insertion sort to parallel

W. Niu et al. (Eds.): ATIS 2015, CCIS 557, pp. 226–237, 2015.
DOI: 10.1007/978-3-662-48683-2_20

execution in spatially constrained networks of devices. To reduce the number of comparators required, the method of modified merge sort is proposed in [10].

Wei et al. [13] employs the Expectation Maximization algorithm to iteratively approximate the optimal data partitioning. A novel periodic multi-round data distribution model is presented and a parallel sorting algorithm for Multisets is proposed by using the characteristics of multi-threading technology and multi-level caches on multi-core architectures [14]. Shirahata et al. [15] propose a MapReduce-based out-of-core GPU memory management technique for processing large-scale graph applications on heterogeneous GPU-based supercomputers. Ye and Huang [16] aims to model transient stability as an objective function rather than an inequality constraint and consider classic transient stability constrained OPF (TSCOPF) as a tradeoff procedure using Pareto ideology.

Cloud computing is a model for enabling ubiquitous network access to a shared pool of configurable computing resources [17]. It is the development of distributed processing, parallel processing and grid computing. It relies on sharing of resources to achieve coherence and economies of scale, similar to the utility over a network. It is a novel approach to share infrastructure faced with ultra-large-scale distributed environment. The core is to provide data storage and network services.

With the rapid development of the amount of data in cloud computing environment, it is particular important to study how to analyze and process these data fast and effectively [9, 11, 12]. How to sort large-scale data efficiently in cloud computing becomes a significant research [18–27]. Whether the used sorting algorithm can achieve high performance and how much time and resource in cloud computing it consumes is concerned widely.

A Large-scale sorting algorithm based on cloud computing makes use of parallel processing characteristics to put the vast amounts of data across distributed computer separately. Combined with the traditional sorting algorithm, it can address the sorting problems more efficiently. It not only excludes the inefficiencies of massive data sorting, but also makes better use of computer resources through parallel processing.

2 Construction of Large-Scale Distributed Sorting Platform

One big advantage of cloud computing is the ability to quickly and efficiently process massive amounts of data. A cloud computing platform can be assembled from a set of distributed machines in different locations, connected to a single network or hub service. This model not only gets rid of the limitations of hardware devices, while extending better to respond quickly to changes in user demand.

Large-scale distributed sorting platform is a system prototype, which aims to providing a range of sorting solutions for cloud computing. The main function of this platform concludes sorting of massive data in cloud, completing the sorting process intelligently, gathering the sorting results. The system operating environment is shown in Fig. 1. Each host counts the number of working machine via a specific port and distributes corresponding data to each one. Each working machine is controlled by host to sort data and order the summary data and request host the next scheduling. Host sends "data request" command to the working machine which has completed the sorting and

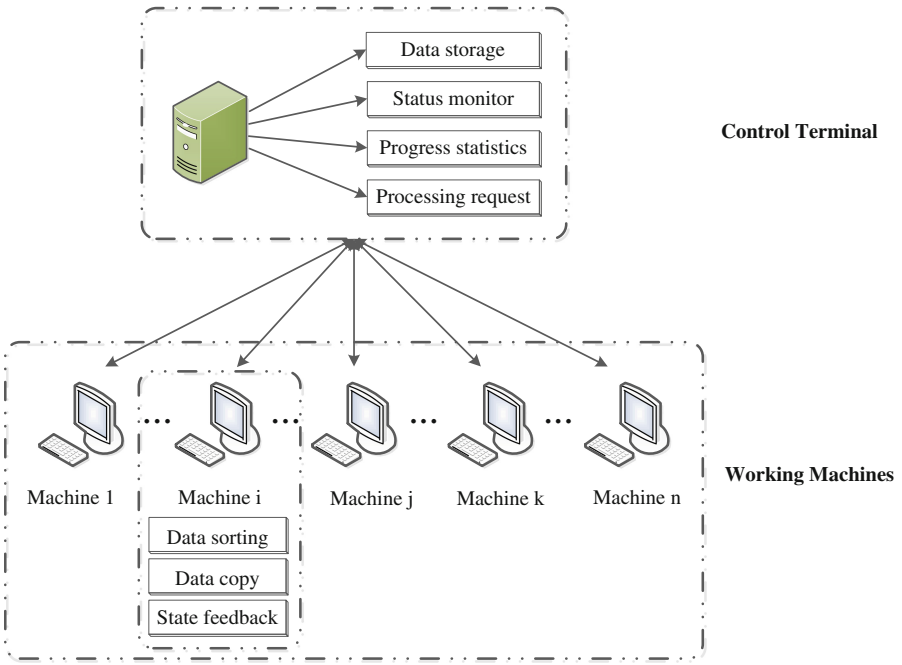

Fig. 1. System operating environment of large-scale distributed sorting platform

enters the status ***Done***. It completes the request then the next round until that the final host confirms that $N - 1$ machines, N is the total number of machines, have entered the status Termination. When received a request ***Done***, all sorting has been completed.

2.1 Business Logic Design

2.1.1 Business Logic Design of Host

Business logic design of host mainly includes these four aspects:

- Data storage: Host of the platform is mainly based on the cloud server. The stored data is the initial mass of data to be sorted and the final sorted data. Host will achieve even distribution and release of data based on the total number of working machine. It achieves hierarchical storage according to business logic in order to reduce the burden of data transfer.
- Status monitor: The host sets up a state parameter for each working machine to monitor complete schedule. The status parameter consists of two values: ***Done*** means working machine has completed data sorting and requests new data. ***Termination*** shows that the work machine enters the status ***Termination***, at the same

time, the host will increase the number of working machine which is in the terminate status.

- Progress statistics: The host sets a number in order to determine the completion of the entire sorting process based on the state feedback of working machine feedback.
- Processing request: The host analyzes the feedback information sent by working machines and makes the next issue work orders. When the host gets the feedback ***Done***, it continues to monitor other working machine. When the working machine B also sends the feedback ***Done***, the host will send the number of B to A. Then A will request data to B.

2.1.2 Business Logic Design of Working Machine

Business logic design of working machine mainly includes these three aspects:

- Data sorting: First, plenty of data inside of the working machine use fast sorting algorithm so that data presents ordering. Second, when two or more work machines complete their own data sorting, a working machine will request data from another working machine and merge the two parts of the data and sort them.
- Data copy: When the working machine ***A*** which is in waiting state requests data from another machine ***B*** which has completed the sorting, B sends the data through a specific port to ***A***. The working machine enters status ***Termination***.
- State feedback: A working machine will send data to the host after completion of its sorting and enters the feedback ***Done***. The working machine will send ***Termination*** to host when it has completed the data sending. Host will record the number of working machine which is in status ***Termination***.

3 Large-Scale Distributed Sorting Algorithm Design

3.1 Large-Scale Distributed Sorting Algorithm Design of Host

3.1.1 Data Distribution Based on Constraint of the Number of Working Machine

The amount of data distributed to each working machine is the key step. If the distribution is any amount of data, the sorting time of each working machine will be different obviously. Some of them may enter status ***Done*** to wait while others may still be sorting, which will lead to a waste of time to balance the working machines.

In our large-scale sorting algorithm, by designing the average distribution mechanism, it can ensure that each data distributed by host is average. The sorting completion time of each working machine is basically the same in order to avoid wasting of resources due to unnecessary waiting. The design ideas are like this:

N represents the number of working machines collected by host through a specific port. The amount of data in the host is *M,* and *data[n] (n = 1, 2, L, N)* represents the data sent to working machine from hosts. The data distribution algorithm based on constraint of working machine is shown in Algorithm 1.

Algorithm 1. Data Distribution Algorithm based on constraint of working machine

Input : N for number of working machines, M for amount of data in the host
Return: data[] for the data send to working machines

```
Begin：

if(M%N == 0)
{
        for (i=0;i<N;i++)
                data[i+1] = M/N;
        endfor
}
else
{
        for(i=0;i<M/N;i++)
                data[i+1] = M/N+1;
        endfor
        for(i=M%N;i<N;i++)
                data[i+1] = M/N;
        endfor
}
endif
```

3.1.2 Task Distribution Based on Constraint of Free Time

Host analyzes the feedback sent by working machines and makes the next issue work orders. Host sets a status parameter ***FlagInHost***, in which the attribute has the following states:

None: No machine is in the waiting state.

Termination: The sort is over.

Wait: Waiting for the rest of the work to complete.

There is a set of numbers ***waitNum*** of the working machine in waiting status. It is used to monitor the progress of sorting in the work machine. The host also sets a statistical attribute ***FinishCount*** to record the number of status ***Termination***.

The host accepts the status request from each working machine. When the working machine A is sending the feedback information ***Done*** to the host, A will be waiting status. If another working machine B is sending the feedback information "***Done***" to the host, the host will send the port number of B to the waiting working machine A. If the request is ***Termination***, the number of the terminated working machine ***FinishCount*** pluses 1, until the ***FinishCount*** is $\boldsymbol{N-1}$. At this point all sort work has been completed. The data distribution algorithm based on constraint of free time is shown in Algorithm 2.

Algorithm 2. *Data Distribution Algorithm based on constraint of free time*

```
Input: inputNum for the number of requesting working machine
Listen(int inputNum, Flag inputFlag)
{
        if ( inputFlag == Termination )
        {
                if( FinishCount == N-1 )
{
        copyFromDataNum(inputNum)
        finish();
}
                else
                        FinishCount++;
                endif
}
elseif ( inputFlag == Done)
{
        if( FlagInHost == NONE )
        {
                FlagInHost = Wait;
                waitNum = inputNum;
}
else
{
        postByPortTo(inputNum,waitNum);
        FlagInHost = None;
}
endif
}
endif
}
```

3.2 Large-Scale Distributed Sorting Algorithm Design of Working Machine

The first time receiving the data host distributing, it will sort their own data quickly. The working machine requests the status ***Done*** to host after completing the sorting. The waiting mechanism is shown in Algorithm 3.

```
selfSort( Elem A[] )
{
        Qsort(A,i,j);
        postByPortTo(HOST,caculatorNum,Done);
        wait();
}
```

Algorithm 3. *Waiting Algorithm*

Input*: cpyNum for the data distribution target*

```
Wait()
{
        int cpyNum = getFromHost();
        Elem B[];
        B = requestDataFrom(selfNum,cpyNum);
        int length = A.length + B.length;
        Elem mergeElem[length];
        for(int i = 0 ,left = 0,right = 0; i<length; i++ )
        {
                if(left>=length)
                        mergeElem[i] = B[right++];
                elseif (right>=length)
                        mergeElem[i] = B[left++];
                elseif(Comp::lt(A[left],B[right]))
                        mergeElem[i]=B[left++];
                else
                        mergeElem[i] = B[right++];
                endif
}
endfor
}
```

When the working machine A enters waiting status, it requests data to working machine B which just completes sorting. Working machine B will send the sorting data to A and send host the status ***Termination***. A will merge the two groups data and send status ***Done*** to host. It re-enters the wait status and repeats the previous step. Running Mechanism is shown in Fig. 2.

4 Implementation and Validation of Algorithm

4.1 Time Complexity

If the amount of data to be sorted is M, and there is N machine for sorting. Then each working machine initial data n is

$$n = M/N \tag{1}$$

Merge sort average time efficiency is

$$t_1 = n \log n \tag{2}$$

When each working machine completes its own data sorting, it needs to request the sorted data in other working machine. And then sort the two parts of the sorted data.

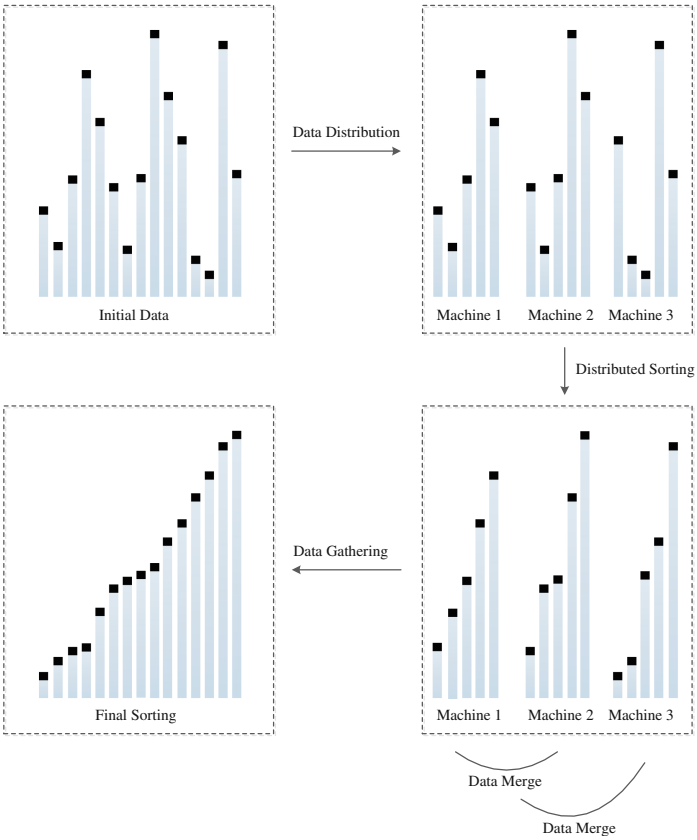

Fig. 2. Running mechanism of large-scale distributed data sorting

Because these two parts of the data have a good order, the two part of the orderly data integration of the time efficiency is

$$t_2 = \Theta(n) \tag{3}$$

Assuming that the time of the host distribution data is constant and the data transferring time is constant too. And it is $\Theta(1)$.

First Step: Host distributes data to the working machines. The work machine achieves the sort of internal data. We set the time consumed for

$$T_1 = \Theta(1) + n \log n \tag{4}$$

Second Step: After the each working machine sort completing, half of the working machines will request data. And then execute merge sort. Its time consume is

$$T_2 = \frac{N}{2}\Theta(1) + n \tag{5}$$

By analogy, we assuming that it executes k rounds, the whole process of sorting is completed.

K Step: There is only one machine requests data from another working machine. And through merging sort, it achieves the entire data sorting. Time consumption is

$$T_k = \frac{N}{2^{k-1}}\Theta(1) + 2^{k-2}n \tag{6}$$

After analysis, we can get

$$2^k = N$$

Simplify formula, it will be

$$k = \log N \tag{7}$$

Total time efficiency is

$$T = T_1 + T_2 + \cdots T_k$$

Thus combined formula (1)–(7), we can get the time complexity as follows.

$$T = \Theta(M + N) + \frac{M}{N}\log\frac{M}{N}$$

4.2 Feasibility and Efficiency

Here, we present the experimental result of our large-scale distributed sorting algorithm based on cloud computing. Our evaluation is divided into two parts feasibility and efficiency. In the process of this algorithm experiment, there is no way through real cloud computing environment to implement the algorithm. So we evaluate the algorithm by some computers. The proposed distributed sorting algorithm is deployed into 11 computers. The computer configuration is roughly equivalent. One of these computers is selected to be host. The rest computers are working machines. The experimental environment is shown in Table 1.

Firstly, we implement a Generator in C++ to generate a specified number of random numbers and store them on the storage of host. We use the Generator to produce a set of random numbers with 100 000, 1 000 000, 100 000 000 as data set. The data set are randomly generated, with no distribution.

We implement Quick Sort, Merge Sort and Distributed Sort in these computers. We evaluate the efficiency by the time consumption. The time consumption is record when the algorithm is starting instead of the testing program is starting. The time will be record until the algorithm completes.

Table 1. Experiment environment

Hardware		
Computer configuration	OS	Win 7
	CPU	Inter(R) Core(TM) i5-3470S(2.9 GHz*2)
	RAM	4 GB
Software		
Port configuration	IP arrange	192.168.1.105-192.168.1.115
	Port	8990
	Transfer protocol	TCP
Data configuration	Generator	Function rand() in C++
	Compiler tool	Visual Studio2010

After that, the program will write the result into a file which is used to compare with other algorithms. We can use the result to evaluate the feasibility and correctness. Repeating the environment above 10 times, and the result is shown in Table 2.

Table 2. Time consumption comparison of algorithms

The amount of data set	Quick-sort	Merge-sort	Large-scale distributed sorting
100 000	0.205000 s	0.367000 s	0.092000 s
1 000 000	12.303000 s	21.223000 s	3.120300 s
100 000 000	1162.017000 s	1011.310300 s	147.230000 s

By comparison, we find that the efficiency of large-scale distributed sorting algorithm based on cloud computing algorithm is significantly higher than common data sorting algorithms when processing on large-scale data set. According to optimize the data request and transmission and the memory resource usage, the time to sort large-scale data decreases about 60 %. When the amount of data is larger, the efficiency gap is more obvious.

5 Conclusion

In this paper, we proposes a large-scale distributed sorting algorithm based on the idea of parallel processing of cloud computing. Through experiments, we found out that the algorithm can be implemented to complete a massive data sorting. And the most effective quick-sort and merge-sort are used in the process of the partial preliminarily sorting of data. It aims to ensure the high efficiency of the algorithm running, the cooperative cooperation of multiple working machines and better use of computer resources. The experiment verified the feasibility and efficiency of the proposed algorithm. In the future, our research mainly includes the following aspects: 1. Optimization of the data request and transmission algorithm. 2. Optimization of memory resource usage.

Acknowledgment. This project is supported by Chinese Academy of Sciences under the project Pilot Special Key Technologies on Information Security, No. Y2W0012306.

References

1. Itani, W., Kayssi, A., Chehab, A.: Privacy as a service: privacy-aware data storage and processing in cloud computing architectures. In: Eighth IEEE International Conference on Dependable, Autonomic and Secure Computing. DASC 2009, pp. 711–716. IEEE (2009)
2. Zhang, S., Zhang, S., Chen, X., Huo, X.: The comparison between cloud computing and grid computing. In: ICCASM 2010 - 2010 International Conference on Computer Application and System Modeling, vol. 11, pp. 1172–1175 (2010)
3. Xia, T., Li, Z., Yu, N.: Research on cloud computing based on deep analysis to typical platforms. In: Jaatun, M.G., Zhao, G., Rong, C. (eds.) Cloud Computing. LNCS, vol. 5931, pp. 601–608. Springer, Heidelberg (2009)
4. Davidson, A., Tarjan, D., Garland, M., et al.: Efficient parallel merge sort for fixed and variable length keys. In: Innovative Parallel Computing (InPar), pp. 1–9. IEEE (2012)
5. Jeon, M., Kim, D.: Parallelizing merge sort onto distributed memory parallel computers. In: Zima, H.P., Joe, K., Sato, M., Seo, Y., Shimasaki, M. (eds.) ISHPC 2002. LNCS, vol. 2327, pp. 25–34. Springer, Heidelberg (2002)
6. Minsoo, J., Dongseung, K.: Parallel merge sort with load balancing. Int. J. Parallel Prog. **31**, 21–33 (2003)
7. Nanjesh, B.R., Tejonidhi, M.R., Rajesh, T.H., et al.: Parallel merge sort based performance evaluation and comparison of MPI and PVM. In: 2013 IEEE Conference on Information and Communication Technologies (ICT), pp. 530–534. IEEE (2013)
8. Max, O.H., Christof, T.: Spatial sorting algorithms for parallel computing in networks. In: Proceedings - 2011 5th IEEE Conference on Self-adaptive and Self-organizing Systems Workshops, pp. 73–78 (2011)
9. Manouchehr Zadahmad, J., Parisa Yousefzadeh, F.: Heuristic and pattern based merge sort. Procedia Comput. Sci. **3**, 322–324 (2011)
10. Chang, R.C.H., Wei, M.F., Chen, H.L., et al.: Implementation of a high-throughput modified merge sort in MIMO detection systems. IEEE Trans. Circ. Syst. I: Regular Papers **61**(9), 2730–2737 (2014)
11. Cérin, C., Koskas, M., Fkaier, H., Jemni, M.: Sequential in-core sorting performance for a SQL data service and for parallel sorting on heterogeneous clusters. Future Gener. Comput. Syst. **22**, 776–783 (2006)
12. Lin, H., Li, C., Wang, Q., Zhao, Y., Pan, N., Zhuang, X., Shao, L.: Automated tuning in parallel sorting on multi-core architectures. In: D'Ambra, P., Guarracino, M., Talia, D. (eds.) Euro-Par 2010, Part I. LNCS, vol. 6271, pp. 14–25. Springer, Heidelberg (2010)
13. Wei, J., Yu, H., Chen, J.H., et al.: Parallel clustering for visualizing large scientific line data. In: 2011 IEEE Symposium on Large Data Analysis and Visualization (LDAV), pp. 47–55. IEEE (2011)
14. Zhong, C., Qu, Z., Yang, F., et al.: Parallel multisets sorting using aperiodic multi-round distribution strategy on heterogeneous multi-core clusters. In: 2010 Third International Symposium on Parallel Architectures, Algorithms and Programming (PAAP), pp. 247–254. IEEE (2010)
15. Shirahata, K., Sato, H., Matsuoka, S.: Out-of-core GPU memory management for MapReduce-based large-scale graph processing. In: 2014 IEEE International Conference on Cluster Computing (CLUSTER), pp. 221–229. IEEE (2014)

16. Ye, C.J., Huang, M.X.: Multi-objective optimal power flow considering transient stability based on parallel NSGA-II. IEEE Trans. Power Syst. **30**(2), 857–866 (2015)
17. Xiao, Z., Xiao, Y.: Security and privacy in cloud computing. IEEE Commun. Surv. Tutorials **15**(2), 843–859 (2013)
18. Olman, V., Mao, F., Wu, H., et al.: Parallel clustering algorithm for large data sets with applications in bioinformatics. IEEE/ACM Trans. Comput. Biol. Bioinform. (TCBB) **6**(2), 344–352 (2009)
19. Scaling up Machine Learning: Parallel and Distributed Approaches. Cambridge University Press (2011)
20. Mehlhorn, K.: Data Structures and Algorithms 1: Sorting and Searching. Springer, Heidelberg (2013)
21. Brin, S., Page, L.: Reprint of: The anatomy of a large-scale hypertextual web search engine. Comput. Netw. **56**(18), 3825–3833 (2012)
22. Rao, S., Ramakrishnan, R., Silberstein, A., et al.: Sailfish: a framework for large scale data processing. In: Proceedings of the Third ACM Symposium on Cloud Computing, p. 4. ACM (2012)
23. Di Martino, A., Yan, C.G., Li, Q., et al.: The autism brain imaging data exchange: towards a large-scale evaluation of the intrinsic brain architecture in autism. Mol. Psychiatry **19**(6), 659–667 (2014)
24. Rasmussen, A., Porter, G., Conley, M., et al.: TritonSort: a balanced large-scale sorting system. In: NSDI (2011)
25. Sakr, S., Liu, A., Batista, D.M., et al.: A survey of large scale data management approaches in cloud environments. IEEE Commun. Surv. Tutorials **13**(3), 311–336 (2011)
26. Cuzzocrea, A., Song, I.Y., Davis, K.C.: Analytics over large-scale multidimensional data: the big data revolution. In: Proceedings of the ACM 14th International Workshop on Data Warehousing and OLAP, pp. 101–104. ACM (2011)
27. Vora, M.N.: Hadoop-HBase for large-scale data. In: 2011 International Conference on Computer Science and Network Technology (ICCSNT), vol. 1, pp. 601–605. IEEE (2011)